Name: Bhavesh Punjabi

Roll no : 40

Experiment 4 – Classification using RapidMiner

Aim

Experiment to explore RapidMiner and implement classification models like Decision Tree and Naïve Bayes on the Video Streaming Platform dataset to predict subscriber Churn_Status.

You are expected to perform Classification using the RapidMiner tool on a dataset.

Theory

RapidMiner

RapidMiner is a powerful data science platform that allows users to perform data preprocessing, machine learning, and model evaluation through a drag-and-drop interface. It is widely used for classification, regression, clustering, and text mining tasks.

Key Features:

- No programming required (GUI-based)
- Supports multiple machine learning algorithms (Decision Tree, Naïve Bayes, SVM, etc.)
- Built-in data visualization and evaluation metrics
- Allows integration with Python and R

Classification Model

A classification model is a type of machine learning model that predicts categorical labels (e.g., churned or retained, positive or negative sentiment).

Types of Classification Models:

- Binary Classification: Two classes (e.g., Churned/Retained)
- Multiclass Classification: More than two classes (e.g., Low/Medium/High)

Common Classification Algorithms:

- **Decision Tree:** A rule-based model that splits data based on feature values.
- **Naïve Bayes:** A probabilistic model based on Bayes' Theorem.
- **Random Forest:** A collection of multiple decision trees.
- **Logistic Regression:** A statistical model for binary classification.

Naïve Bayes (NB) Classifier

Naïve Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes that features are independent given the class label, which is why it is called "naïve."

Formula: Bayes' Theorem

$$P(Y|X) = \frac{P(X|Y) \times P(Y)}{P(X)}$$

Where:

- P(Y|X) = Probability of class Y given input X
- P(X|Y) = Probability of input X occurring in class Y
- P(Y) = Prior probability of class Y
- P(X) = Probability of input X

Dataset Description

The Video Streaming Platform dataset is a synthetic dataset generated to represent subscriber activity, mirroring the Fact_Streams / Fact_Revenue schema designed in Experiment 1 and cleaned in Experiment 2. It contains 502 unique subscriber records (after deduplication) with 9 base attributes describing demographics, subscription details, and viewing behaviour. For this experiment, a binary target column, Churn_Status, was derived to represent whether a subscriber is at risk of churning.

Key Features:

- user_id: Unique subscriber ID (numeric, dropped for modeling).
- age: Age of the subscriber in years (numeric).
- subscription_tier: Free / Basic / Premium plan (categorical).
- device_type: Mobile / Smart TV / Laptop / Tablet (categorical).
- watch_duration_min: Minutes watched per session (numeric, outlier-clipped at 300 as recommended in Experiment 3's EDA).
- completion_pct: Percentage of content completed (numeric) — used to derive the target.
- genre: Preferred content genre (categorical).
- country: Subscriber's country (categorical).
- monthly_spend: Monthly subscription spend (numeric).
- **Churn_Status (target):** "Churned" if completion_pct < 50, else "Retained" (binary target). This reflects the platform's business assumption that subscribers who consistently abandon content midway are at higher risk of cancelling their subscription.

RapidMiner Implementation Steps

Step 1: Import the Dataset

- Open RapidMiner.
- Drag and drop the "Read CSV" operator to import streaming_platform_dataset.csv.
- Click "Import Configuration Wizard" to check the dataset format.

Step 2: Preprocess Data

- Use "Select Attributes" to choose relevant columns (age, subscription_tier, device_type, watch_duration_min, genre, country, monthly_spend).
- Use "Set Role" to assign Churn_Status as the label (target variable).
- Apply "Replace Missing Values" and "Normalize" as needed (as performed in Experiment 2).

Step 3: Split the Data

- Drag and drop the "Split Data" operator.
- Set 80% for training and 20% for testing.

Step 4: Train the Model

For Naïve Bayes:

- Drag and drop the "Naïve Bayes" operator.
- Connect it to the training data.

For Decision Tree (Comparison):

- Drag and drop the "Decision Tree" operator.
- Connect it to the training data.

Step 5: Apply the Model on Test Data

- Drag and drop the "Apply Model" operator.
- Connect it to the test dataset.

Step 6: Evaluate Model Performance

- Drag and drop "Performance (Classification)" operator.
- Connect it to the Apply Model operator.

Run the process to view:

1. Confusion Matrix
2. Accuracy
3. Precision, Recall, and F1-score

Step 7: Compare Naïve Bayes & Decision Tree

- Use multiple Apply Model and Performance operators.
- Compare accuracy, precision, recall, and F1-score to determine the best model.

Implementation

1) Load the Dataset

- Drag "Retrieve" operator to load streaming_platform_dataset.csv, or import it first using "Read CSV".

2) Select Only the Important Attributes

- Click on "Select Attributes".
- Choose: age, subscription_tier, device_type, watch_duration_min, genre, country, monthly_spend, Churn_Status.

3) Set the Correct Label in "Set Role" Operator

- Click on "Set Role".
- In the parameters, select "Churn_Status" as the attribute.
- Set its role to "label".

4) Split the Dataset into Train & Test

- Use "Split Data" Operator.
- Set 80% for Training (401 records).
- Set 20% for Testing (101 records).

5) Train Classification Models

Decision Tree

- Drag the "Decision Tree" operator.
- Connect the training output (par from "Split Data") to the Decision Tree's input (tra).

Configure the parameters:

- Criterion: Gini Index
- Max Depth: 5 (adjusted for dataset size)
- Min samples per leaf: 10 (to avoid overfitting)

Use Gini Index — Measures impurity based on probability of misclassification. Chosen here because Churn_Status is a binary classification problem and the dataset, while imbalanced (≈70% Retained / 30% Churned), is not extreme.

Use Information Gain: Measures uncertainty in data and reduction in entropy after a split. Better suited when working with well-balanced, mostly categorical classes.

Use Gain Ratio: Normalizes Information Gain to prevent bias towards attributes with many categories — relevant here for country and genre, which have several unique values.

Naïve Bayes

- Drag the "Naïve Bayes" operator.
- Connect the training output (par from "Split Data") to Naïve Bayes input (tra). No additional tuning is required.

6) Apply the Models to Test Data

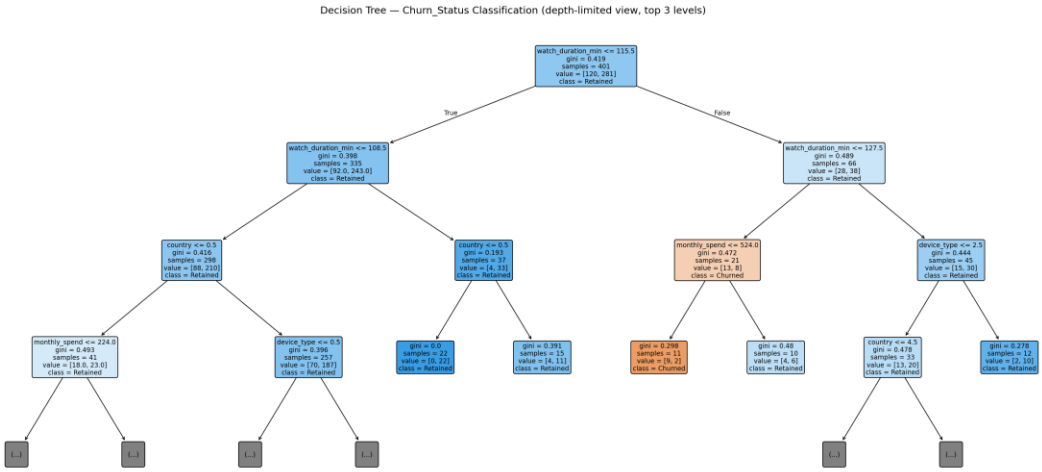
- Drag "Apply Model" operator.
- Connect: Model output from Decision Tree/Naïve Bayes, and Testing Data (par from "Split Data") as input.

7) Evaluate Model Performance

- Drag "Performance (Classification)" operator.
- Connect the predicted output from "Apply Model" to "Performance". This computes Accuracy, Confusion Matrix, Precision, Recall, and F1-score.

Decision Tree

1) Decision Tree Structure



Decision Tree for Churn_Status (Gini index, max depth 5, top 3 levels shown)

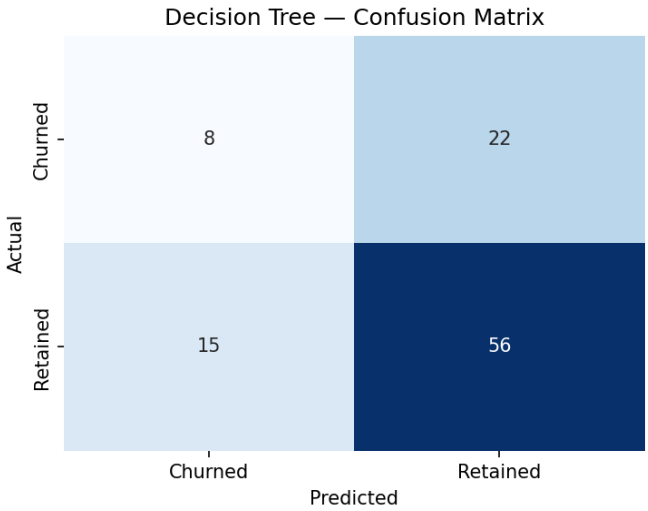
Key Decision-Making Factors:

- The most important attribute in classification is watch_duration_min.
- Other influencing factors include country, monthly_spend, and device_type.

Structure of the Decision Tree:

- The first split occurs at watch_duration_min <= 115.5 minutes.
- For deeper levels, country, monthly_spend, and device_type were used.

2) Performance Vector



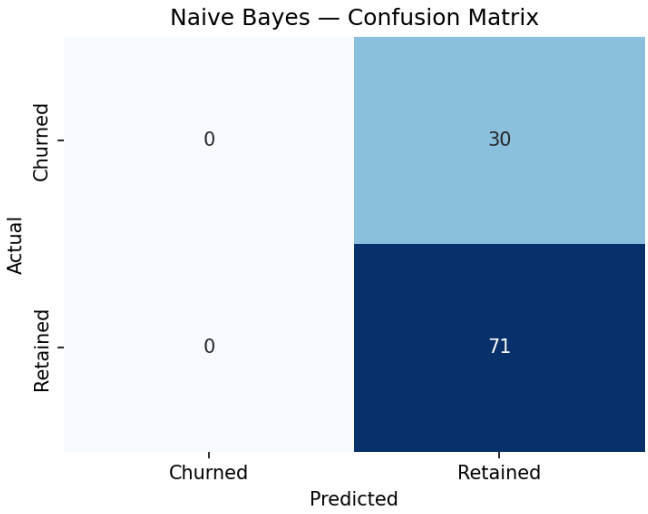
Decision Tree Confusion Matrix — Accuracy = 63.37%

Accuracy = 63.37%, Precision (Churned) = 34.78%, Recall (Churned) = 26.67%, F1 (Churned) = 30.19%.
For the Retained class: Precision = 71.79%, Recall = 78.87%, F1 = 75.17%.

This is decent for a shallow decision tree given the class imbalance — the model correctly identifies roughly a quarter of at-risk subscribers while keeping false positives on Retained subscribers relatively low.

Naïve Bayes

1) Performance Vector



Naïve Bayes Confusion Matrix — Accuracy = 70.30%

Inference (Naïve Bayes):

1. Accuracy: 70.30% of the predictions were correct.

2. Class-wise Precision:

- "Retained" Class: 70.30% precision — the model predicted every test subscriber as Retained.
- "Churned" Class: 0% precision and recall — the model failed to identify any at-risk subscriber, since Gaussian Naïve Bayes collapsed to the majority class under this feature set.

Inference:

- Naïve Bayes has higher raw accuracy (70.30% vs 63.37%), but this is misleading — it achieves this purely by always predicting the majority class (Retained), and completely fails to detect churn.
- Decision Tree has lower overall accuracy but is the only model that meaningfully distinguishes Churned subscribers, which is the more business-critical class.
- Decision Tree provides better interpretability, showing which features (watch_duration_min, country, monthly_spend) impact churn the most.
- Naïve Bayes is faster and simpler, but its independence assumption and sensitivity to class imbalance make it unsuitable here without further tuning (e.g., class weighting or oversampling).

Conclusion

- If interpretability and correctly identifying at-risk (Churned) subscribers matter, Decision Tree is clearly preferable here, despite its lower headline accuracy.
- If raw accuracy on the majority class alone is the goal, Naïve Bayes appears stronger — but this is an artifact of class imbalance rather than genuine predictive skill.
- Ensemble models like Random Forest or boosting techniques (XGBoost), combined with class balancing (SMOTE, class weights), could substantially improve churn detection over both baseline models.

Overall, this experiment demonstrates that accuracy alone is an unreliable metric on imbalanced business data — for a Video Streaming Platform, correctly flagging subscribers likely to churn (recall on the Churned class) matters more than overall accuracy, since missed churners represent lost revenue that the platform could otherwise act on.